

# FastAPI

## Complete Learning Guide

*From Beginner to Associate Software Engineer (Full Stack)  
L1 & L2 Interview Preparation*

| What's inside                                     |  |
|---------------------------------------------------|--|
| ✓ Python prerequisites & async fundamentals       |  |
| ✓ FastAPI core: routing, validation, dependencies |  |
| ✓ Pydantic v2, SQLAlchemy 2.0, Alembic            |  |
| ✓ Authentication: OAuth2 + JWT + password hashing |  |
| ✓ Async DB, middleware, background tasks          |  |
| ✓ WebSockets, file uploads, streaming             |  |
| ✓ Testing with pytest & TestClient                |  |
| ✓ Docker, deployment, CI/CD basics                |  |
| ✓ Full-stack integration (React + FastAPI)        |  |
| ✓ 80+ L1 & L2 interview questions with answers    |  |
| ✓ System design + coding round prep               |  |

# Table of Contents

## Part 1: Foundations

1. Python Prerequisites for FastAPI
2. Async & Await Fundamentals
3. Type Hints & Pydantic Basics

## Part 2: FastAPI Core

4. Getting Started with FastAPI
5. Path & Query Parameters
6. Request Body & Response Models
7. Validation & Error Handling
8. Dependency Injection

## Part 3: Building Real APIs

9. Database Integration (SQLAlchemy 2.0)
10. Authentication & Authorization (JWT)
11. Middleware & CORS
12. Background Tasks & Celery
13. WebSockets & Real-time
14. File Uploads & Streaming

## Part 4: Production & Full Stack

15. Testing FastAPI Apps
16. Logging, Caching & Rate Limiting
17. Docker & Deployment
18. Full Stack: React + FastAPI

## Part 5: Interview Preparation

19. L1 Interview Questions (40+)
20. L2 Interview Questions (40+)
21. Coding Challenges & System Design
22. Final Checklist & Resources

# Part 1: Foundations

Before diving into FastAPI, you need a solid grasp of modern Python features that FastAPI heavily relies on: type hints, `async/await`, decorators, and Pydantic. This part will get you ready.

## 1. Python Prerequisites for FastAPI

### 1.1 Why these prerequisites matter

FastAPI is built on three modern Python pillars: **type hints** (for validation and docs), **async/await** (for high concurrency), and **decorators** (for declarative routing). Mastering these three concepts is 60% of becoming productive in FastAPI.

### 1.2 Setting up your environment

Always use a virtual environment per project. Recommended Python version: 3.10+.

```
# Create and activate a virtual environment
python -m venv venv

# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate

# Install FastAPI with the optional standard dependencies
pip install "fastapi[standard]"

# This installs: fastapi, uvicorn, python-multipart,
# email-validator, httpx, jinja2, and the CLI tools.
```

### 1.3 Type hints crash course

Type hints tell Python (and FastAPI) what type each variable, parameter, or return should be. FastAPI uses them to validate requests, serialize responses, and auto-generate OpenAPI docs.

```
from typing import Optional, List, Dict, Union

# Basic type hints
name: str = "Ada"
age: int = 30
height: float = 5.6
is_active: bool = True

# Collections
tags: list[str] = ["python", "fastapi"]      # Python 3.9+
scores: dict[str, int] = {"math": 90}

# Optional and Union
nickname: Optional[str] = None               # str or None
identifier: Union[int, str] = "abc"           # int or str
identifier: int | str = "abc"                 # Python 3.10+ syntax
```

```
# Function signatures
def greet(name: str, times: int = 1) -> str:
    return f"Hello {name}! " * times
```

## 1.4 Decorators recap

A decorator is a function that wraps another function to extend its behavior without modifying it. FastAPI uses decorators like `@app.get("/")` to register routes.

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Done: {func.__name__}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

print(add(2, 3))
# Calling add
# Done: add
# 5
```

## 1.5 Context managers and the 'with' statement

FastAPI uses context managers for resource cleanup (DB sessions, file handles). The `contextlib` module provides utilities to build your own.

```
from contextlib import contextmanager

@contextmanager
def db_session():
    print("open session")
    try:
        yield "session_object"
    finally:
        print("close session")

with db_session() as session:
    print(f"using {session}")
```

## 2. Async & Await Fundamentals

FastAPI is an **ASGI** (Asynchronous Server Gateway Interface) framework. Understanding async is critical to write performant endpoints and to ace interview questions.

### 2.1 Sync vs Async

Synchronous code blocks the thread while waiting (e.g., for a network response). Asynchronous code yields control back to the event loop during those waits, allowing other tasks to run. Result: a single process can handle thousands of concurrent I/O-bound requests.

```
import asyncio
import time

# Sync version: takes 6 seconds total (3 + 3)
def fetch_sync(name):
    time.sleep(3)
    return f"{name} done"

# Async version: takes ~3 seconds total
async def fetch_async(name):
    await asyncio.sleep(3)
    return f"{name} done"

async def main():
    results = await asyncio.gather(
        fetch_async("task1"),
        fetch_async("task2"),
    )
    print(results)

asyncio.run(main())
```

### 2.2 When to use async def vs def in FastAPI

Rule of thumb: use `async def` when your endpoint awaits something (async DB calls, httpx requests). Use regular `def` for blocking/CPU-bound work — FastAPI will run it in a thread pool automatically. Never mix blocking I/O (e.g., `requests.get`) inside an `async def` handler — it will freeze the event loop.

### 2.3 Common async patterns

```
import asyncio
import httpx

async def fetch_url(client, url):
    response = await client.get(url)
    return response.status_code

async def fetch_all(urls):
    async with httpx.AsyncClient() as client:
        tasks = [fetch_url(client, url) for url in urls]
        return await asyncio.gather(*tasks)

# Run many requests concurrently
urls = ["https://example.com"] * 10
results = asyncio.run(fetch_all(urls))
```

## 3. Type Hints & Pydantic Basics

Pydantic is a data validation library built on type hints. FastAPI uses Pydantic v2 to validate request bodies, serialize responses, and generate JSON Schema for OpenAPI docs.

### 3.1 Your first Pydantic model

```
from pydantic import BaseModel, Field, EmailStr
from datetime import datetime

class User(BaseModel):
    id: int
    name: str = Field(min_length=2, max_length=50)
    email: EmailStr
    age: int = Field(ge=0, le=150)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    is_active: bool = True

# Create from dict (e.g., JSON body)
data = {"id": 1, "name": "Ada", "email": "ada@example.com", "age": 36}
user = User(**data)

# Serialize back
print(user.model_dump())      # dict
print(user.model_dump_json()) # JSON string
```

### 3.2 Nested models and lists

```
from pydantic import BaseModel
from typing import Optional

class Address(BaseModel):
    street: str
    city: str
    zip_code: str

class User(BaseModel):
    name: str
    email: str
    address: Address
    tags: list[str] = []
    profile: Optional[dict] = None

user_data = {
    "name": "Ada",
    "email": "ada@example.com",
    "address": {
        "street": "1 Lovelace Ln",
        "city": "London",
        "zip_code": "EC1A 1AA"
    },
    "tags": ["admin", "founder"]
}
user = User(**user_data)
```

### 3.3 Validators and computed fields

```
from pydantic import BaseModel, field_validator, computed_field
```

```
class Product(BaseModel):
    name: str
    price: float
    quantity: int

    @field_validator("price")
    @classmethod
    def price_must_be_positive(cls, v):
        if v <= 0:
            raise ValueError("Price must be positive")
        return v

    @computed_field
    @property
    def total_value(self) -> float:
        return self.price * self.quantity

p = Product(name="Book", price=15.0, quantity=3)
print(p.total_value) # 45.0
```

# Part 2: FastAPI Core

## 4. Getting Started with FastAPI

### 4.1 What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs based on standard Python type hints. Key features: automatic OpenAPI/Swagger docs, data validation through Pydantic, async support, dependency injection, and performance comparable to Node.js and Go (thanks to Starlette and Uvicorn).

### 4.2 Hello World

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="My First API", version="1.0.0")

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Run with:

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000

# Open in browser:
#   http://localhost:8000          -> your endpoint
#   http://localhost:8000/docs    -> Swagger UI
#   http://localhost:8000/redoc   -> ReDoc
```

### 4.3 HTTP methods (GET, POST, PUT, PATCH, DELETE)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items")
def list_items():
    return [{"id": 1, "name": "pen"}]

@app.post("/items", status_code=201)
def create_item(item: dict):
    return {"created": item}

@app.put("/items/{item_id}")
def replace_item(item_id: int, item: dict):
    return {"id": item_id, "data": item}

@app.patch("/items/{item_id}")
def update_item(item_id: int, item: dict):
    return {"id": item_id, "updated": item}
```



```

@app.delete("/items/{item_id}", status_code=204)
def delete_item(item_id: int):
    return None

```

## 4.4 Project structure for real apps

```

my_app/
├── app/
│   ├── __init__.py
│   ├── main.py                # FastAPI instance
│   └── core/
│       ├── config.py          # Settings (env vars)
│       ├── security.py        # Auth helpers
│       └── api/
│           ├── deps.py        # Shared dependencies
│           └── v1/
│               ├── routes/
│               │   ├── users.py
│               │   └── items.py
│               └── api.py      # APIRouter aggregator
│   ├── models/                # SQLAlchemy models
│   ├── schemas/                # Pydantic schemas
│   ├── crud/                   # DB operations
│   └── db/
│       ├── base.py
│       └── session.py
├── tests/
├── alembic/
├── .env
├── requirements.txt
└── Dockerfile

```

## 5. Path & Query Parameters

### 5.1 Path parameters

```
from fastapi import FastAPI, Path

app = FastAPI()

# Simple path parameter
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

# With validation using Path()
@app.get("/items/{item_id}")
def get_item(
    item_id: int = Path(..., ge=1, le=1000, description="The ID of the item")
):
    return {"item_id": item_id}

# Path parameters with enums
from enum import Enum

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
def get_model(model_name: ModelName):
    return {"model": model_name.value}
```

### 5.2 Query parameters

```
from fastapi import Query
from typing import Optional

@app.get("/products")
def list_products(
    skip: int = 0,
    limit: int = 10,
    search: Optional[str] = None,
    sort_by: str = Query("name", regex="^(name|price|date)$"),
    tags: list[str] = Query(default=[])
):
    # /products?skip=20&limit=5&search=shoe&tags=red&tags=large
    return {
        "skip": skip,
        "limit": limit,
        "search": search,
        "sort_by": sort_by,
        "tags": tags
    }
```

### 5.3 Required vs optional

A parameter without a default value is **required**. Use `= None` or `Optional[str] = None` to make it optional. Use `Query(...)` (Ellipsis) to make a query param required while still adding validation.

## 5.4 Combining path, query, and body

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float

@app.put("/items/{item_id}")
def update_item(
    item_id: int,                # path
    item: Item,                  # body
    q: Optional[str] = None,    # query
):
    return {"item_id": item_id, "q": q, "item": item}
```

## 6. Request Body & Response Models

### 6.1 Request body with Pydantic

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()

class ItemCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=100)
    description: Optional[str] = Field(None, max_length=500)
    price: float = Field(..., gt=0)
    tax: Optional[float] = Field(None, ge=0)

@app.post("/items")
def create_item(item: ItemCreate):
    total = item.price + (item.tax or 0)
    return {"**item.model_dump()", "total": total}
```

### 6.2 Response models

Use `response_model` to filter and validate outgoing data. This is critical for security (e.g., never returning password hashes).

```
class UserCreate(BaseModel):
    username: str
    email: str
    password: str    # accepted as input

class UserPublic(BaseModel):
    id: int
    username: str
    email: str        # password NOT included

@app.post("/users", response_model=UserPublic, status_code=201)
def create_user(user: UserCreate):
    # save to DB ...
    return {
        "id": 1,
        "username": user.username,
        "email": user.email,
        "password": user.password    # will be stripped by response_model
    }
```

### 6.3 Status codes and headers

```
from fastapi import status, Response

@app.post("/items", status_code=status.HTTP_201_CREATED)
def create(item: ItemCreate, response: Response):
    response.headers["X-Custom-Header"] = "abc"
    return item
```

### 6.4 Multiple body parameters and Body()

```
from fastapi import Body
from pydantic import BaseModel
```

```

class Item(BaseModel):
    name: str
    price: float

class User(BaseModel):
    username: str

@app.put("/items/{item_id}")
def update(
    item_id: int,
    item: Item,
    user: User,
    importance: int = Body(..., ge=1, le=5)
):
    return {"item_id": item_id, "item": item, "user": user, "importance": importance}

# Expected body:
# {
#   "item": {...},
#   "user": {...},
#   "importance": 3
# }

```

## 6.5 Pydantic Settings (12-factor config)

```

# core/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    APP_NAME: str = "MyAPI"
    DEBUG: bool = False
    DATABASE_URL: str
    SECRET_KEY: str
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60

settings = Settings()

# Then use anywhere:
# from app.core.config import settings

```

## 7. Validation & Error Handling

### 7.1 Automatic validation

FastAPI returns a structured JSON error response (HTTP 422) with details about every invalid field, automatically. No work required.

### 7.2 Raising HTTPException

```
from fastapi import FastAPI, HTTPException, status

app = FastAPI()
db = {1: {"name": "Alice"}}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id not in db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"User {user_id} not found",
            headers={"X-Error": "UserMissing"}
        )
    return db[user_id]
```

### 7.3 Custom exception handlers

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

class ItemOutOfStock(Exception):
    def __init__(self, sku: str):
        self.sku = sku

app = FastAPI()

@app.exception_handler(ItemOutOfStock)
async def out_of_stock_handler(request: Request, exc: ItemOutOfStock):
    return JSONResponse(
        status_code=409,
        content={"error": "out_of_stock", "sku": exc.sku}
    )

@app.post("/orders")
def create_order(sku: str):
    if sku == "X123":
        raise ItemOutOfStock(sku=sku)
    return {"ordered": sku}
```

### 7.4 Overriding the validation error response

```
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content={
```

```
        "message": "Validation failed",  
        "errors": exc.errors()  
    }  
)
```

## 8. Dependency Injection

Dependency injection is FastAPI's superpower. Use it for shared logic: getting DB sessions, checking authentication, applying pagination, etc. Anything declared as a dependency runs before your endpoint and its result is passed in.

### 8.1 Function dependencies

```
from fastapi import Depends, FastAPI
from typing import Optional

app = FastAPI()

def pagination_params(
    skip: int = 0,
    limit: int = 10,
    q: Optional[str] = None
):
    return {"skip": skip, "limit": limit, "q": q}

@app.get("/items")
def list_items(pagination: dict = Depends(pagination_params)):
    return pagination

@app.get("/users")
def list_users(pagination: dict = Depends(pagination_params)):
    return pagination
```

### 8.2 Class-based dependencies

```
class Pagination:
    def __init__(self, skip: int = 0, limit: int = 10):
        self.skip = skip
        self.limit = limit

@app.get("/items")
def list_items(p: Pagination = Depends()):
    return {"skip": p.skip, "limit": p.limit}
```

### 8.3 Sub-dependencies

```
def get_token_header(x_token: str = Header(...)):
    if x_token != "secret":
        raise HTTPException(403, "Invalid token")
    return x_token

def get_current_user(token: str = Depends(get_token_header)):
    # decode token, fetch user
    return {"username": "alice"}

@app.get("/me")
def read_me(user: dict = Depends(get_current_user)):
    return user
```

### 8.4 Yield dependencies (setup & teardown)

```
from sqlalchemy.orm import Session
from app.db.session import SessionLocal
```



```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users/{user_id}")
def read_user(user_id: int, db: Session = Depends(get_db)):
    return db.query(User).filter(User.id == user_id).first()
```

## 8.5 Global dependencies

```
app = FastAPI(dependencies=[Depends(verify_api_key)])

# Or on a specific router:
router = APIRouter(
    prefix="/admin",
    dependencies=[Depends(require_admin)]
)
```

# Part 3: Building Real APIs

## 9. Database Integration (SQLAlchemy 2.0)

### 9.1 Setting up SQLAlchemy

```
pip install sqlalchemy[asyncio] psycpg2-binary alembic asyncpg

# app/db/session.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

SQLALCHEMY_DATABASE_URL = "postgresql://user:pass@localhost/mydb"
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass
```

### 9.2 Defining models

```
# app/models/user.py
from sqlalchemy import String, Integer, Boolean, DateTime, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from datetime import datetime
from app.db.session import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(120), unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255))
    is_active: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

    items: Mapped[list["Item"]] = relationship(back_populates="owner")

class Item(Base):
    __tablename__ = "items"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    owner: Mapped["User"] = relationship(back_populates="items")
```

### 9.3 Pydantic schemas (input vs output)

```
# app/schemas/user.py
from pydantic import BaseModel, EmailStr, ConfigDict
from datetime import datetime

class UserBase(BaseModel):
    email: EmailStr

class UserCreate(UserBase):
    password: str
```

```
class UserRead(UserBase):
    id: int
    is_active: bool
    created_at: datetime
    # Allow returning a SQLAlchemy ORM object directly
    model_config = ConfigDict(from_attributes=True)
```

## 9.4 CRUD operations

```
# app/crud/user.py
from sqlalchemy.orm import Session
from app.models.user import User
from app.schemas.user import UserCreate
from app.core.security import hash_password

def get_user(db: Session, user_id: int) -> User | None:
    return db.query(User).filter(User.id == user_id).first()

def get_user_by_email(db: Session, email: str) -> User | None:
    return db.query(User).filter(User.email == email).first()

def list_users(db: Session, skip: int = 0, limit: int = 100):
    return db.query(User).offset(skip).limit(limit).all()

def create_user(db: Session, data: UserCreate) -> User:
    user = User(email=data.email, hashed_password=hash_password(data.password))
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

def delete_user(db: Session, user_id: int) -> bool:
    user = get_user(db, user_id)
    if not user:
        return False
    db.delete(user)
    db.commit()
    return True
```

## 9.5 Endpoint wiring

```
# app/api/v1/routes/users.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app.api.deps import get_db
from app.schemas.user import UserCreate, UserRead
from app.crud import user as crud_user

router = APIRouter(prefix="/users", tags=["users"])

@router.post("", response_model=UserRead, status_code=201)
def create_user(payload: UserCreate, db: Session = Depends(get_db)):
    if crud_user.get_user_by_email(db, payload.email):
        raise HTTPException(409, "Email already registered")
    return crud_user.create_user(db, payload)

@router.get("/{user_id}", response_model=UserRead)
def read_user(user_id: int, db: Session = Depends(get_db)):
    user = crud_user.get_user(db, user_id)
    if not user:
        raise HTTPException(404, "User not found")
    return user
```

## 9.6 Alembic migrations

```
# Init Alembic
alembic init alembic

# alembic/env.py - point to your Base.metadata
from app.db.session import Base
from app.models import user, item # import all model modules
target_metadata = Base.metadata

# Generate a migration
alembic revision --autogenerate -m "create users and items"

# Apply migrations
alembic upgrade head

# Roll back one step
alembic downgrade -1
```

## 9.7 Async SQLAlchemy (advanced)

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
from sqlalchemy import select

engine = create_async_engine("postgresql+asyncpg://user:pass@localhost/db")
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session

@app.get("/users/{user_id}")
async def read_user(user_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == user_id))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(404)
    return user
```

# 10. Authentication & Authorization (JWT)

## 10.1 Hash passwords with bcrypt

```
pip install "passlib[bcrypt]" python-jose[cryptography]

# app/core/security.py
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)
```

## 10.2 JWT token creation & decoding

```
from datetime import datetime, timedelta, timezone
from jose import jwt, JWTError
from app.core.config import settings

ALGORITHM = "HS256"

def create_access_token(subject: str, expires_minutes: int = 60) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=expires_minutes)
    payload = {"sub": subject, "exp": expire}
    return jwt.encode(payload, settings.SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str) -> str | None:
    try:
        data = jwt.decode(token, settings.SECRET_KEY, algorithms=[ALGORITHM])
        return data.get("sub")
    except JWTError:
        return None
```

## 10.3 Login endpoint

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from app.api.deps import get_db
from app.crud.user import get_user_by_email
from app.core.security import verify_password, create_access_token

router = APIRouter(tags=["auth"])

@router.post("/login")
def login(
    form: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db)
):
    user = get_user_by_email(db, form.username) # OAuth2 uses 'username'
    if not user or not verify_password(form.password, user.hashed_password):
        raise HTTPException(
            status.HTTP_401_UNAUTHORIZED,
            "Incorrect email or password",
            headers={"WWW-Authenticate": "Bearer"}
        )
```

```
token = create_access_token(subject=user.email)
return {"access_token": token, "token_type": "bearer"}
```

## 10.4 Protected routes & current user

```
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
):
    email = decode_token(token)
    if email is None:
        raise HTTPException(401, "Invalid token")
    user = get_user_by_email(db, email)
    if not user:
        raise HTTPException(401, "User not found")
    return user

@router.get("/me", response_model=UserRead)
def read_me(current: User = Depends(get_current_user)):
    return current
```

## 10.5 Role-based authorization

```
def require_role(role: str):
    def checker(user: User = Depends(get_current_user)):
        if user.role != role:
            raise HTTPException(403, "Insufficient permissions")
        return user
    return checker

@router.delete("/users/{id}")
def delete_user(
    id: int,
    admin: User = Depends(require_role("admin")),
    db: Session = Depends(get_db)
):
    ...
```

# 11. Middleware & CORS

## 11.1 CORS middleware

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com", "http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

## 11.2 Custom middleware

```
import time
from fastapi import Request

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start
    response.headers["X-Process-Time"] = f"{duration:.3f}"
    print(f"{request.method} {request.url.path} -> {response.status_code} ({duration:.3f}s)")
    return response
```

## 11.3 GZip and Trusted Hosts

```
from fastapi.middleware.gzip import GZipMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware

app.add_middleware(GZipMiddleware, minimum_size=1000)
app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=["api.example.com", "*.example.com"]
)
```

# 12. Background Tasks & Celery

## 12.1 Built-in BackgroundTasks

Use `BackgroundTasks` for lightweight, fire-and-forget work that runs after the response is returned (e.g., sending a welcome email).

```
from fastapi import BackgroundTasks

def send_welcome_email(email: str):
    print(f"Sending email to {email}...")
    # ... actual SMTP call
```

```

@app.post("/signup")
def signup(email: str, background_tasks: BackgroundTasks):
    # create user ...
    background_tasks.add_task(send_welcome_email, email)
    return {"status": "ok"}

```

## 12.2 Celery for heavy/distributed work

```

# tasks.py
from celery import Celery

celery_app = Celery(
    "worker",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/1",
)

@celery_app.task
def process_video(path: str):
    # Long-running encoding job
    ...

# In FastAPI endpoint:
from tasks import process_video

@app.post("/upload")
def upload(path: str):
    task = process_video.delay(path)
    return {"task_id": task.id}

# Start worker (terminal):
# celery -A tasks worker --loglevel=info

```



## 13. WebSockets & Real-time

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

class ConnectionManager:
    def __init__(self):
        self.active: list[WebSocket] = []

    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.active.append(ws)

    def disconnect(self, ws: WebSocket):
        self.active.remove(ws)

    async def broadcast(self, message: str):
        for conn in self.active:
            await conn.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/{username}")
async def websocket_endpoint(ws: WebSocket, username: str):
    await manager.connect(ws)
    await manager.broadcast(f"{username} joined")
    try:
        while True:
            data = await ws.receive_text()
            await manager.broadcast(f"{username}: {data}")
    except WebSocketDisconnect:
        manager.disconnect(ws)
        await manager.broadcast(f"{username} left")
```

Browser client example:

```
const ws = new WebSocket("ws://localhost:8000/ws/alice");
ws.onmessage = (event) => console.log(event.data);
ws.onopen = () => ws.send("hello");
```

## 14. File Uploads & Streaming

### 14.1 Single and multiple file uploads

```
from fastapi import File, UploadFile
from typing import List
import shutil

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    with open(f"./uploads/{file.filename}", "wb") as buf:
        shutil.copyfileobj(file.file, buf)
    return {
        "filename": file.filename,
        "content_type": file.content_type,
        "size_kb": round(file.size / 1024, 2)
    }

@app.post("/upload-many")
```

```
async def upload_many(files: List[UploadFile] = File(...)):
    return [{"filename": f.filename, "size": f.size} for f in files]
```

## 14.2 Streaming responses

```
from fastapi.responses import StreamingResponse

@app.get("/download")
def download():
    def iter_file():
        with open("largefile.zip", "rb") as f:
            while chunk := f.read(8192):
                yield chunk
    return StreamingResponse(
        iter_file(),
        media_type="application/zip",
        headers={"Content-Disposition": "attachment; filename=archive.zip"}
    )
```

# Part 4: Production & Full Stack

## 15. Testing FastAPI Apps

### 15.1 TestClient basics

```
# tests/test_users.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_user():
    response = client.post(
        "/users",
        json={"email": "test@example.com", "password": "secret123"}
    )
    assert response.status_code == 201
    data = response.json()
    assert data["email"] == "test@example.com"
    assert "id" in data

def test_get_user_not_found():
    response = client.get("/users/999")
    assert response.status_code == 404
```

### 15.2 Pytest fixtures & test DB override

```
# conftest.py
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.main import app
from app.db.session import Base
from app.api.deps import get_db

TEST_DB_URL = "sqlite:///./test.db"
engine = create_engine(TEST_DB_URL, connect_args={"check_same_thread": False})
TestingSession = sessionmaker(bind=engine)

@pytest.fixture
def db_session():
    Base.metadata.create_all(bind=engine)
    session = TestingSession()
    try:
        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=engine)

@pytest.fixture
def client(db_session):
    def override_get_db():
        try:
            yield db_session
        finally:
            pass
    app.dependency_overrides[get_db] = override_get_db
```

```
yield TestClient(app)
app.dependency_overrides.clear()
```

## 15.3 Testing async endpoints

```
# pip install pytest-asyncio httpx
```

```
import pytest
from httpx import AsyncClient, ASGITransport
from app.main import app

@pytest.mark.asyncio
async def test_root():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        response = await ac.get("/")
        assert response.status_code == 200
```

# 16. Logging, Caching & Rate Limiting

## 16.1 Structured logging

```
import logging
import sys

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
    stream=sys.stdout,
)

logger = logging.getLogger("api")

@app.get("/items/{id}")
def get_item(id: int):
    logger.info("Fetching item %s", id)
    return {"id": id}
```

## 16.2 Redis caching

```
# pip install redis
import json, redis
r = redis.Redis(host="localhost", port=6379, decode_responses=True)

@app.get("/products/{pid}")
def get_product(pid: int):
    cache_key = f"product:{pid}"
    cached = r.get(cache_key)
    if cached:
        return json.loads(cached)
    product = expensive_db_call(pid)
    r.setex(cache_key, 300, json.dumps(product)) # 5 min TTL
    return product
```

## 16.3 Rate limiting (slowapi)

```
# pip install slowapi
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from fastapi import Request

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.get("/limited")
@limiter.limit("5/minute")
def limited(request: Request):
    return {"ok": True}
```

# 17. Docker & Deployment

## 17.1 Production Dockerfile

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

# System deps
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential && rm -rf /var/lib/apt/lists/*

# Python deps
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# App code
COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

## 17.2 docker-compose for local dev

```
# docker-compose.yml
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgresql://app:app@db:5432/app
      SECRET_KEY: dev-secret
    depends_on: [db, redis]
    volumes:
      - ./app
    command: uvicorn app.main:app --host 0.0.0.0 --reload

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    ports: ["5432:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7
    ports: ["6379:6379"]

volumes:
  pgdata:
```

## 17.3 Gunicorn + Uvicorn workers

```
gunicorn app.main:app \
  -w 4 \
  -k uvicorn.workers.UvicornWorker \
  --bind 0.0.0.0:8000 \
  --timeout 60 \
```

--access-logfile -

## 17.4 Cloud options

Common production stacks: AWS ECS Fargate, AWS Lambda + API Gateway (with Mangum), Google Cloud Run, Azure Container Apps, Render, Fly.io, Railway, Heroku, or a plain VM with Nginx as reverse proxy. For Kubernetes, deploy as a regular HTTP service behind an Ingress. Always put a reverse proxy in front for TLS termination and to enforce request timeouts.

# 18. Full Stack: React + FastAPI

## 18.1 The full-stack architecture

A typical full-stack project: React (or Vue / Angular) talks to FastAPI over HTTP/JSON. FastAPI manages authentication, business logic, and the database. Both run independently in containers and are deployed behind a reverse proxy.

## 18.2 Calling FastAPI from React

```
// api.js
const API_URL = import.meta.env.VITE_API_URL;

export async function login(email, password) {
  const body = new URLSearchParams({ username: email, password });
  const res = await fetch(`${API_URL}/login`, {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body,
  });
  if (!res.ok) throw new Error("Login failed");
  return res.json(); // { access_token, token_type }
}

export async function fetchMe(token) {
  const res = await fetch(`${API_URL}/me`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  return res.json();
}
```

## 18.3 React component using FastAPI

```
// LoginPage.jsx
import { useState } from "react";
import { login, fetchMe } from "./api";

export default function LoginPage() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [user, setUser] = useState(null);
  const [error, setError] = useState("");

  async function handleSubmit(e) {
    e.preventDefault();
    try {
      const { access_token } = await login(email, password);
      localStorage.setItem("token", access_token);
      const me = await fetchMe(access_token);
      setUser(me);
    } catch (err) {
      setError(err.message);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={(e) => setEmail(e.target.value)} />
      <input type="password" value={password} />
    </form>
  );
}
```



```

        onChange={(e) => setPassword(e.target.value)} />
        <button type="submit">Login</button>
        {user && <p>Welcome {user.email}</p>}
        {error && <p style={{ color: "red" }}>{error}</p>}
    </form>
  );
}

```

## 18.4 Auto-generating a TypeScript client from OpenAPI

```

# Generate a fully typed JS/TS client from FastAPI's openapi.json
npx @hey-api/openapi-ts \
  -i http://localhost:8000/openapi.json \
  -o src/client \
  -c @hey-api/client-fetch

```

# Now you get autocompletion, types, and zero hand-written API code.

## 18.5 Serving the React build from FastAPI (single deploy)

```

from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

# API routes first
app.include_router(api_router, prefix="/api/v1")

# Serve React build at root (must be last)
app.mount("/", StaticFiles(directory="frontend/dist", html=True), name="frontend")

```

# Part 5: Interview Preparation

## 19. L1 Interview Questions (40+)

L1 questions test foundational understanding. Expect them in screening interviews and at the start of a technical round. Answer in 1–3 short sentences.

### Q: What is FastAPI?

A: FastAPI is a modern, high-performance Python web framework for building APIs based on standard type hints. It uses Starlette for the web parts and Pydantic for data validation, and supports async/await natively.

### Q: Why is FastAPI fast?

A: It's built on Starlette (ASGI) and Uvicorn, supports async I/O so a single worker can handle thousands of concurrent connections, and uses Pydantic v2 (compiled in Rust) for fast validation/serialization.

### Q: What is ASGI? How is it different from WSGI?

A: ASGI (Asynchronous Server Gateway Interface) is the async successor to WSGI. ASGI supports long-lived connections like WebSockets and async request handling; WSGI is synchronous and one-request-per-thread.

### Q: What server do you use to run FastAPI?

A: Uvicorn (ASGI server) for development. In production, Gunicorn with Uvicorn workers, or Hypercorn, behind a reverse proxy like Nginx.

### Q: What is Pydantic and why does FastAPI use it?

A: Pydantic is a data validation library that uses Python type hints. FastAPI uses it to validate request bodies, serialize responses, and generate OpenAPI/JSON Schema docs automatically.

### Q: Difference between path parameters and query parameters?

A: Path parameters are part of the URL path (`/items/{id}`) and identify a specific resource. Query parameters appear after '?' (`/items?skip=0&limit=10`) and filter or modify the request.

### Q: How does FastAPI generate Swagger docs?

A: It introspects your route handlers, parameters, and Pydantic models to build an OpenAPI 3.x JSON spec, then serves Swagger UI at `/docs` and ReDoc at `/redoc` by default.

### Q: What HTTP status code should a successful POST return?

A: 201 Created when a new resource is created, or 200 OK / 204 No Content otherwise depending on whether you return data.

### Q: How do you return a custom status code?

A: Set `status_code` on the decorator: `@app.post("/items", status_code=201)`, or modify the Response object.

### Q: How do you make a query parameter required?

A: Use `Query(...)` (Ellipsis) or simply don't give it a default value: `def get(name: str = Query(...))`.

**Q: What is dependency injection in FastAPI?**

A: A way to declare reusable logic (DB session, auth check, pagination) that FastAPI runs before your endpoint and injects as a parameter using `Depends()`.

**Q: How do you handle errors in FastAPI?**

A: Raise `HTTPException(status_code, detail)` for known errors, or register custom handlers with `@app.exception_handler(MyException)`.

**Q: What's the difference between `async def` and `def` in FastAPI?**

A: `async def` runs in the event loop and should await non-blocking I/O. `def` runs in a thread pool — FastAPI handles it automatically. Never use blocking I/O inside `async def`.

**Q: What is `OAuth2PasswordBearer`?**

A: A FastAPI security utility that declares 'this endpoint expects a Bearer token' for OpenAPI and extracts it from the Authorization header for use in dependencies.

**Q: What is a JWT?**

A: JSON Web Token: a signed, base64-encoded JSON payload used for stateless authentication. It contains claims (sub, exp, etc.) and a signature so the server can verify it without a session store.

**Q: How do you hash passwords?**

A: With a slow, salted hash like `bcrypt` or `argon2` via `passlib`. Never store plain text or use MD5/SHA-1.

**Q: How do you connect FastAPI to PostgreSQL?**

A: Use `SQLAlchemy` + `psycopg2` (sync) or `asyncpg` (async). Create an engine, a `SessionLocal`, and inject a session via `Depends(get_db)`.

**Q: What is Alembic?**

A: Alembic is `SQLAlchemy`'s migration tool. It tracks schema changes as versioned scripts so you can evolve your database safely in production.

**Q: How do you enable CORS?**

A: Add `CORSMiddleware` with `allow_origins`, `allow_methods`, and `allow_headers`.

**Q: What is `APIRouter`?**

A: A way to group related routes in their own module, then include them in the main app with `app.include_router(router, prefix="/users", tags=["users"])`.

**Q: How do you upload a file in FastAPI?**

A: Use `UploadFile = File(...)`. `UploadFile` is a spooled file object you can read async or copy to disk.

**Q: How do you serve static files?**

A: Mount `StaticFiles`: `app.mount("/static", StaticFiles(directory="static"))`.

**Q: What's the difference between `PUT` and `PATCH`?**

A: PUT replaces the whole resource (send all fields). PATCH applies a partial update (send only changed fields).

**Q: What is `response_model` and why use it?**

A: It defines the schema for the response. FastAPI filters the data through it, dropping any extra fields — vital for hiding sensitive data like password hashes.

**Q: How do you add custom headers to a response?**

A: Inject the Response object and set `response.headers["X-Custom"] = "value"`.

**Q: What is `BackgroundTasks`?**

A: A built-in FastAPI class for fire-and-forget tasks that run after the response is sent — good for lightweight things like sending emails.

**Q: When would you use Celery instead?**

A: For long-running, distributed, retryable jobs (video processing, ML inference, batch jobs) that need persistence and workers separate from the API.

**Q: How do you test a FastAPI app?**

A: Use `TestClient` from `fastapi.testclient` with `pytest`. Override dependencies via `app.dependency_overrides` for DB.

**Q: How do you configure environment variables?**

A: Use `pydantic-settings` with a Settings class that reads from a `.env` file or OS environment.

**Q: What is Starlette?**

A: The lightweight ASGI framework FastAPI is built on. It provides routing, middleware, WebSockets, and the request/response cycle.

**Q: Difference between sync and async DB drivers?**

A: Sync drivers (`psycopg2`) block the thread. Async drivers (`asyncpg`) integrate with the event loop so your async endpoints stay non-blocking.

**Q: What is the purpose of `yield` in a dependency?**

A: It separates setup from teardown: code before `yield` runs before the request, code after `yield` runs after the response is sent (used for DB session close).

**Q: How do you version an API?**

A: Mount routers under prefixes: `/api/v1`, `/api/v2`. Keep v1 stable; add new features in v2.

**Q: What is OpenAPI?**

A: An open standard for describing HTTP APIs (formerly Swagger). FastAPI generates an `openapi.json` automatically that tools use for docs and client generation.

**Q: How do you customize the docs URL?**

A: 

```
FastAPI(docs_url="/swagger", redoc_url="/api-docs",
openapi_url="/api.json").
```

**Q: What does the 422 status code mean in FastAPI?**

A: Unprocessable Entity — returned automatically when Pydantic validation fails on the request body, path, or query parameters.

**Q: How do you read JSON request body?**

A: Declare a parameter typed as a Pydantic BaseModel: `def create(item: Item)`. FastAPI parses and validates it.

**Q: How do you read form data?**

A: Use `Form(...)` on each parameter and `pip install python-multipart`.

**Q: What is the difference between BaseModel and dataclass?**

A: Both define data containers. BaseModel adds runtime validation, JSON Schema, serialization, and is what FastAPI understands for request/response models.

**Q: How do you redirect a user?**

A: Return `RedirectResponse(url="/login", status_code=302)`.

**Q: What is the default port for Uvicorn?**

A: Port 8000.

**Q: How do you stream a large file?**

A: Return `StreamingResponse` with a generator that yields chunks; alternatively use `FileResponse` for static files on disk.

## 20. L2 Interview Questions (40+)

L2 questions go deeper into design, trade-offs, and production concerns. Answers should demonstrate you understand the *why*, not just the *how*.

### Q: Walk me through what happens when a request hits FastAPI.

A: Uvicorn (ASGI server) receives the raw HTTP request and passes an ASGI scope dict to Starlette. Starlette routes to the matching path operation. FastAPI resolves dependencies (DB session, auth), validates path/query/body using Pydantic, calls the handler (in event loop if async, in threadpool if sync), serializes the return value through `response_model`, runs middleware on the way out, and Uvicorn writes the HTTP response.

### Q: How does FastAPI achieve near-Node.js performance?

A: Three things: an async I/O model (event loop), a thin routing layer (Starlette), and Pydantic v2 with a Rust core for validation/serialization. Together they minimize per-request CPU and avoid thread-context switches.

### Q: Explain the dependency injection system in depth.

A: Dependencies are callables that FastAPI resolves before invoking the endpoint. The DI graph is topologically sorted: a dep can require sub-deps, which require theirs, etc. Each unique dep is resolved once per request (caching by default), and `yield-deps` wrap the endpoint in a context manager so teardown runs after the response is sent.

### Q: How do you implement refresh tokens?

A: Issue two JWTs at login: a short-lived access token (15 min) and a long-lived refresh token (7–30 days), stored in an `HttpOnly` cookie. Expose `POST /refresh` that validates the refresh token, optionally rotates it, and returns a new access token. Track refresh tokens in DB so you can revoke them.

### Q: How do you secure JWTs?

A: Use a strong `SECRET_KEY` (32+ bytes), set short expirations, validate `exp/iat/iss/aud` claims, use HS256 or RS256 (never 'none'), transport over HTTPS only, store refresh tokens `HttpOnly+Secure+SameSite`, and implement a token revocation list (DB or Redis) for logout/compromise.

### Q: Sync vs async endpoint — what determines the choice?

A: Async wins when the endpoint waits on I/O you can await (async DB driver, `httpx`, `Redis-py`, `asyncio`). Sync is fine — and safer — for CPU work or libraries with no async equivalent. Mixing blocking calls inside async def is the single biggest performance footgun.

### Q: How do you avoid the N+1 query problem in SQLAlchemy?

A: Use eager loading: `selectinload`, `joinedload`, or `subqueryload` on relationships. For example: `session.execute(select(User).options(selectinload(User.items)))`.

### Q: Explain database connection pooling.

A: A pool reuses a fixed number of physical connections across requests, avoiding the cost of opening TCP+TLS+auth handshakes per query. SQLAlchemy's `QueuePool` is default. Size it to  $(\text{max workers}) \times (\text{connections per worker})$ , staying below your DB's `max_connections`.

### Q: Async SQLAlchemy session — how does it differ?

A: You use `AsyncSession`, `async_sessionmaker`, an `async` engine, and `await session.execute(...)`. Lazy loading is disabled by default; you must eager-load. Transactions and commits are all awaitable.

**Q: How do you scale a FastAPI app horizontally?**

A: Run multiple stateless worker processes behind a load balancer. Move session state and caches to Redis. Use a managed Postgres with connection pooling (PgBouncer). Auto-scale based on CPU/RPS. Offload long work to Celery.

**Q: What is the GIL and does it affect FastAPI?**

A: The Global Interpreter Lock prevents multiple threads from running Python bytecode in parallel. For I/O-bound async endpoints it's irrelevant — the event loop is single-threaded and waits release the GIL. For CPU-bound work, use multiprocessing or worker processes (Gunicorn -w).

**Q: Explain rate limiting strategies.**

A: Fixed window (simple, allows bursts at boundaries), sliding window (smoother but more memory), token bucket (allows controlled bursts — most common in APIs), and leaky bucket. Implement in Redis with INCR+EXPIRE or use `slowapi` / `fastapi-limiter`.

**Q: How do you cache API responses?**

A: Use Redis with a key derived from the URL and relevant query/auth. Set a TTL. Invalidate on writes (cache-aside pattern). For per-user data, include `user_id` in the key. Be careful with cache stampedes — use locks or 'stale-while-revalidate'.

**Q: Idempotency — what is it and how do you implement it?**

A: An idempotent endpoint produces the same result when called multiple times. GET/PUT/DELETE are idempotent by definition; POST usually isn't. Use an Idempotency-Key header: store the key + response in Redis for 24h and return the cached response on retries.

**Q: How do you structure a large FastAPI codebase?**

A: Separate by domain (users, items, orders), with each domain having models, schemas, crud, and routes. Keep API routes under `/api/v1` via `APIRouters`. Put cross-cutting concerns (config, security, deps) in a `core/` module. Tests mirror the app structure.

**Q: What's the difference between SQLAlchemy ORM and Core?**

A: Core is the SQL expression language: you build `SELECT/INSERT/etc.` as Python objects. ORM sits on top and maps Python classes to tables. SQLAlchemy 2.0 unifies them: the same `select()` works in both.

**Q: How would you implement role-based access control?**

A: Add a role column on users (or many-to-many to a roles table). Write a dependency factory `require_role("admin")` that depends on the current user and raises 403 if the role doesn't match. For fine-grained perms, use a permissions table and check by permission name.

**Q: How do you handle file uploads larger than memory?**

A: `UploadFile` is spooled — it switches to a temp file on disk past a size threshold. Stream chunks with `await file.read(8192)` in a loop. For very large uploads use multipart with resume support (e.g., tus protocol) or pre-signed S3 URLs so the API never touches the bytes.

**Q: What is Pydantic v2 and what changed from v1?**

A: v2 has a Rust core (pydantic-core) that's 5–50× faster. The API changed: BaseModel uses `model_dump`, `model_validate`, `model_config`; validators use `@field_validator` and `@model_validator`; configuration via `ConfigDict`; `from_attributes` replaces `orm_mode`.

**Q: How do you debug slow endpoints?**

A: Add timing middleware. Use SQLAlchemy echo or query logging. Profile with py-spy or cProfile. Check for N+1 queries, missing indexes, blocking I/O in async, and external service latency. APM tools: Datadog, New Relic, Sentry Performance.

**Q: Explain WebSockets vs Server-Sent Events vs long polling.**

A: Long polling: client repeatedly polls; simple but inefficient. SSE: server pushes events over a single HTTP connection, one-directional, auto-reconnect. WebSockets: full duplex, low overhead, ideal for chat/games. Use SSE for notifications, WebSockets for interactive real-time.

**Q: How do you implement pagination?**

A: Offset/limit pagination (`?skip=0&limit=10`) is simple but slow on large tables. Cursor (keyset) pagination uses an indexed sort key — pass a `?after=id` and `SELECT WHERE id > after ORDER BY id LIMIT N` — much faster on big tables.

**Q: How does FastAPI's `response_model` strip fields?**

A: After your endpoint returns, FastAPI validates and serializes the output using the `response_model`'s Pydantic schema. Fields not on the schema are dropped. Use `response_model_exclude_unset=True` for partial responses.

**Q: How do you write integration tests with real Postgres?**

A: Spin up a Postgres in CI (docker-compose, testcontainers, or pytest-postgresql). Apply Alembic migrations in a fixture. Wrap each test in a transaction and roll back at the end so tests don't interfere — or recreate the schema per test class.

**Q: Explain how middleware runs in FastAPI.**

A: Middleware wraps the request/response cycle like an onion. The outermost runs first on the way in and last on the way out. Middleware can short-circuit (return without calling next), modify the request, or modify the response.

**Q: Difference between `BaseHTTPMiddleware` and pure ASGI middleware?**

A: `BaseHTTPMiddleware` is convenient (uses Request/Response objects) but has performance overhead and subtle issues with streaming. Pure ASGI middleware (callable receiving scope, receive, send) is faster and works correctly for all ASGI features.

**Q: What is CSRF and does it apply to FastAPI APIs?**

A: CSRF tricks a logged-in user's browser into making unwanted requests using cookies. JSON APIs called with explicit Bearer tokens (Authorization header) are immune. If you use cookie auth, add CSRF tokens (double-submit cookie pattern) and `SameSite=Lax/Strict` cookies.

**Q: How do you handle migrations with zero downtime?**

A: Use the expand-and-contract pattern: (1) deploy code that reads old + new schema, (2) run migration to add new columns (nullable, no defaults that lock), (3) backfill, (4) deploy code that uses new schema, (5) drop old columns. Never rename — add new + migrate + remove old.

**Q: How do you implement search?**



A: For small datasets: SQL LIKE / ILIKE. For real text search in Postgres: tsvector + GIN index + to\_tsquery. For complex/large search: dedicated engines (Elasticsearch, Meilisearch, Typesense) — FastAPI proxies queries and combines results.

**Q: How would you design a webhook system?**

A: Register subscribers (URL + secret). On event, push a row to an outbox table in the same transaction as the data change (transactional outbox). A worker reads outbox, POSTs to subscribers with HMAC signature, retries with exponential backoff, and dead-letters after N failures.

**Q: Single deployment vs separate backend/frontend?**

A: Separate: independent scaling, deploys, and teams; needs CORS and an API gateway. Single (FastAPI serving the React build): simpler for small apps, no CORS, easier auth via cookies, but couples release cycles.

**Q: What does Uvicorn vs Hypercorn vs Gunicorn give you?**

A: Uvicorn: ASGI server, lightning fast, no built-in process manager. Hypercorn: ASGI server with HTTP/2 and HTTP/3 support. Gunicorn: process manager — typically used as 'gunicorn -k uvicorn.workers' to get Uvicorn's speed plus Gunicorn's worker management.

**Q: How do you log structured JSON logs?**

A: Use the standard logging module with a JSON formatter (python-json-logger, structlog). Include request\_id, user\_id, path, status, duration. Aggregate in ELK, Loki, or a cloud logging service.

**Q: Explain the difference between 401 and 403.**

A: 401 Unauthorized: the request is missing or has invalid credentials. 403 Forbidden: the credentials are valid but the user isn't allowed to perform this action.

**Q: How does FastAPI handle CPU-bound tasks?**

A: Sync def endpoints run in a threadpool, freeing the event loop. For heavy CPU work, offload to a process pool (concurrent.futures), a worker queue (Celery, RQ), or break the work into chunks with await asyncio.sleep(0) to yield.

**Q: How would you implement a soft-delete?**

A: Add a deleted\_at timestamp column. Override your read queries to filter deleted\_at IS NULL. Implement DELETE endpoints to set the timestamp instead of removing rows. Optionally use SQLAlchemy events to enforce automatically.

**Q: How do you handle distributed tracing?**

A: Use OpenTelemetry: instrument FastAPI, SQLAlchemy, httpx, and Redis. Propagate trace context via the W3C traceparent header. Export to Jaeger, Tempo, or a vendor (Datadog APM, Honeycomb).

**Q: Why might you choose FastAPI over Django REST Framework?**

A: FastAPI: better async, faster, type-driven validation, auto OpenAPI, lighter. DRF: batteries-included (admin, ORM, auth, sessions out of the box), mature ecosystem. Choose FastAPI for API-first microservices, DRF for full Django apps.

**Q: How does Pydantic handle datetime serialization?**

A: datetime is serialized to ISO 8601 strings. Customize with field\_serializer or model\_config. For timezone safety, always store and compare in UTC and convert at the edge.

**Q: How do you secure secrets in production?**

A: Never commit them. Use a secrets manager (AWS Secrets Manager, GCP Secret Manager, Vault) or your platform's env-var encryption. Rotate regularly. Inject at container start, not bake into images.

**Q: How do you write a graceful shutdown?**

A: Use the lifespan context manager (FastAPI 0.93+). On shutdown: stop accepting new requests, wait for in-flight ones to finish (with a deadline), close DB pools, flush logs/metrics. Uvicorn handles SIGTERM by default.

**Q: What's an N+1 query and how do you detect it?**

A: Loading N related objects with N separate queries instead of 1 join. Detect with SQLAlchemy event listeners that count queries per request, with django-silk-style middleware, or by inspecting `echo=True` output in dev.

# 21. Coding Challenges & System Design

## 21.1 Common coding round tasks

Expect to live-code a small CRUD service in 30–45 minutes. Practice these patterns until they're muscle memory.

### Task 1: Build a 'Notes' API with auth

```
# Requirements
# - POST /signup, POST /login (JWT)
# - GET/POST/PUT/DELETE /notes (user can only access own)
# - Pagination on list

# Tip – outline first, then fill in:
# 1) main.py + include_router(auth), include_router(notes)
# 2) models (User, Note)
# 3) schemas (UserCreate, UserRead, NoteCreate, NoteRead)
# 4) deps (get_db, get_current_user)
# 5) crud (notes scoped by owner_id)
# 6) routes (auth, notes)
```

### Task 2: Rate-limited public endpoint

```
# Implement /search?q=... that:
# - validates q (min 2 chars)
# - hits external API via httpx (async)
# - caches result in Redis for 60s
# - rate-limits 10 req/min per IP

from fastapi import FastAPI, HTTPException, Request
import httpx, json, redis.asyncio as redis

app = FastAPI()
r = redis.Redis(host="localhost", decode_responses=True)

async def rate_limit(ip: str):
    key = f"rl:{ip}"
    count = await r.incr(key)
    if count == 1:
        await r.expire(key, 60)
    if count > 10:
        raise HTTPException(429, "Too many requests")

@app.get("/search")
async def search(q: str, request: Request):
    if len(q) < 2:
        raise HTTPException(400, "q too short")
    await rate_limit(request.client.host)

    cache_key = f"search:{q}"
    cached = await r.get(cache_key)
    if cached:
        return json.loads(cached)

    async with httpx.AsyncClient(timeout=5) as client:
        resp = await client.get(f"https://api.example.com/?q={q}")
        data = resp.json()

    await r.setex(cache_key, 60, json.dumps(data))
    return data
```

## Task 3: Real-time chat with WebSockets

Build a `/ws/{room}` endpoint that joins users to a room and broadcasts messages. Bonus: persist messages to Postgres and replay the last 50 on join.

## 21.2 System Design Question Walkthrough

### 'Design a URL shortener API like Bit.ly using FastAPI.'

#### Step 1 — Clarify requirements

- Functional: shorten a URL, redirect short → long, view stats per link.
- Scale assumption: 100M URLs total, 1K writes/sec, 10K reads/sec at peak.
- Non-functional: low latency on redirect (<50ms p99), high availability.

#### Step 2 — API contract

```
POST /shorten      { "url": "...", }  -> { "short": "abc123" }
GET  /{code}      -> 301 redirect
GET  /{code}/stats -> { "clicks": N }
```

#### Step 3 — Storage choice

- Postgres for the canonical URL table (short\_code PK, url, owner\_id, created\_at).
- Redis cache for short\_code → url (hot reads).
- Click events to a separate analytics table or stream (Kafka → ClickHouse) — never block redirects on analytics writes.

#### Step 4 — Short code generation

- Option A: base62-encode an auto-incrementing ID (gives ordered, dense codes — but predictable).
- Option B: random 7-char base62 code, retry on collision (unpredictable, slight write overhead).
- For 100M URLs, 7 chars of base62  $\approx$  3.5T combinations — collision probability is negligible.

#### Step 5 — Read path

```
GET /{code}:
1. Look up code in Redis -> if hit, redirect (1 hop).
2. On miss, query Postgres, cache for 1 hour, redirect.
3. Fire-and-forget analytics event (BackgroundTasks or Kafka).
```

#### Step 6 — Scaling & reliability

- Stateless FastAPI workers behind an ALB — autoscale on CPU/RPS.
- Read replicas for Postgres; writes go to primary.
- Cache shard or use Redis Cluster for read fan-out.
- Rate-limit POST /shorten per API key to prevent abuse.
- Bloom filter in front of Postgres to fast-reject unknown codes (optional optimization).

## 21.3 Other system design questions to practice

- Design a file upload service (S3 pre-signed URLs vs through-API).
- Design a chat backend (WebSocket fan-out, message ordering, offline queue).

- Design a job queue (in-process vs Celery vs SQS).
- Design rate limiting that works across multiple API instances.
- Design an OAuth2 server that issues tokens to third-party apps.
- Design a notifications service (email, push, in-app).

## 22. Final Checklist & Resources

### 22.1 Pre-interview checklist

- I can create a FastAPI app from scratch with auth + DB in under 30 minutes.
- I can explain async/await, the event loop, and when not to use async def.
- I can write Pydantic v2 models with validators and computed fields.
- I can set up SQLAlchemy 2.0 (sync + async) and run Alembic migrations.
- I can implement JWT auth with bcrypt and protected routes.
- I can write yield-based dependencies and explain when they run.
- I can write a TestClient test and override dependencies for isolation.
- I can write a Dockerfile and docker-compose for FastAPI + Postgres + Redis.
- I can debug an N+1 query and fix it with selectinload.
- I can explain the request lifecycle from Uvicorn down to the handler.
- I can implement rate limiting and explain token-bucket vs fixed-window.
- I can wire a React frontend to FastAPI with login + protected fetches.
- I have one personal project deployed (Render, Railway, Fly, or AWS).
- I can talk for 3 minutes about a trade-off I made in my project.

### 22.2 Recommended resources

#### Official:

- FastAPI docs — [fastapi.tiangolo.com](https://fastapi.tiangolo.com) (read the whole tutorial section).
- Pydantic v2 docs — [docs.pydantic.dev](https://docs.pydantic.dev).
- SQLAlchemy 2.0 docs — [docs.sqlalchemy.org](https://docs.sqlalchemy.org).
- Starlette docs — for middleware and WebSocket internals.

#### Books / long-form:

- Building Python Microservices with FastAPI — Sherwin John C. Tragura.
- High Performance Python — for async, profiling, and concurrency models.
- Designing Data-Intensive Applications — for the system-design half.

#### Practice projects (build all three):

- Notes API with JWT auth, deployed to Render with a React frontend.
- URL shortener with Redis caching and analytics.
- Real-time chat with WebSockets, Postgres history, and online presence.

### 22.3 Interview-day tips

- When asked an open question, restate it in your own words to confirm.
- Outline the structure before coding — talk about what files / routes / models you'd create.
- Type hints in every snippet, even on a whiteboard. It signals 'I think in types.'

- Mention error handling explicitly: 'I'd raise a 404 here if the user wasn't found.'
- If you don't know something, say 'I haven't done it but here's how I'd approach it.'
- End each answer by asking 'Would you like me to go deeper on any part?'

### **Good luck with your L1 and L2 interviews!**

Remember: depth on a few things beats shallow knowledge of everything. Pick the topics in this guide you're weakest on, build something with them, and explain it out loud. That's how you go from reading FastAPI to *knowing* FastAPI.